COMS5022A: Programming for Data Scientists

HR Analytics Project

Due date: 10 April 2026, 23:59

1 Project Overview

In this project, you will design and implement an HR analytics system that processes employee data, identifies data quality issues, cleans the data, and calculates key HR metrics. This project combines practical programming skills with theoretical analysis of programming techniques.

1.1 Project Goals

1. Design an HR analytics system that can clean and analyse employee data
2. Write a comprehensive technical report explaining your implementation

1.2 Project Structure

This project has two main components:

- **Practical Component (80%):** Implement data cleaning and metric calculation functions
- **Theoretical Component (20%):** Write a detailed report explaining your approach

1.3 Provided files

On Moodle, you will find a file called **project.zip**. Download this file and unzip it to start the project. In the zip file, you will find the following files:

- **cleaning.py**: The file where you will complete the code for Part A.
- **metrics.py**: The file where you will complete the code for Part B.
- **test_cleaning.py**: The file you will use to test your code for Part A.
- **test_metrics.py**: The file you will use to test your code for Part B.
- **uncleaned_dataset.csv**: The file which contains the data you can use to manually test your code for Part A.
- **cleaned_dataset.csv**: The file which contains the data you can use to manually test your code for Part B.
- **TEST_README.pdf**: The file which contains an overview of the testing suite.

2 Academic Integrity

A firm reminder that there is a zero-tolerance policy regarding plagiarism or cheating in the School.

3 Data Format and Column Mapping

The HR dataset is provided as a CSV file with the following columns. The data will be provided to your functions as a 2D list (list of lists), where each inner list represents one employee record. You will receive skeleton code that reads the CSV file and converts it to this format, skipping the header row.

Index	Column Name	Data Type	Description
0	Employee_ID	String	Unique employee identifier (e.g., EMP1000)
1	Department	String	Department name (engineering, sales, hr, marketing, product)
2	Gender	String	Employee gender (Male, Female, Non-binary)
3	Age	Integer	Employee age in years
4	Salary	Float	Annual salary in currency units
5	Training_Hours	Integer	Number of training hours completed
6	Performance_Rating	Float	Performance rating (0.0 to 5.0)
7	Overtime_Ratio	Float	Ratio of overtime hours to regular hours
8	Status	String	Employment status (Active or Resigned)
9	Hire_Date	String	Date of hire (YYYY-MM-DD format)

Below is an example of what an element(data entry) in the list would look like:

```
['EMP1000', 'HR', 'Male', 46, 140000.0, 31, 3.7, 0.23, 'Resigned', '
    2015-04-01']
```

4 Practical Component

4.1 Preface: Code and Comments

Pay attention to the mark distribution for each question. It is structured as $x + y = total$, where x represents the mark for code working and y represents the comments for the code implementation. Marks for comments are awarded for effective and helpful comments, not just the presence of comments. For reference on how to write effective comments, refer to **Chapter 3, Section 3.1.1** in the course notes.

4.2 Part A: Data Cleaning (32+8=40)

Data quality is critical for analytics. Your first task is to implement four data cleaning functions. Each function modifies the dataset in place (modifies the list directly) and returns nothing.

4.2.1 Remove Null Salaries (8+2=10)

Write a function that takes the data set as an argument, removes entries that have invalid salaries, and returns the data set without the invalid entries. All invalid salaries will have a $'nan'$ value.

Sample Input

```
[['EMP1000', 'HR', 'Male', 46, 140000.0, 31, 3.7, 0.23, 'Resigned', '
   2015-04-01'],
 ['EMP1001', 'Engineering', 'Male', 45, nan, 16, 0.7, 0.08, 'Resigned'
    , '2016-10-01'],
 ['EMP1002', 'Marketing', 'Non-binary', 30, 90000.0, 34, 2.5, 0.39, '
    Active', '2016-12-01'],
 ['EMP1003', 'HR', 'Non-binary', 37, 90000.0, 30, 3.3, 0.01, 'Active',
     '2023-02-01']]
```

Sample Output

```
[['EMP1000', 'HR', 'Male', 46, 140000.0, 31, 3.7, 0.23, 'Resigned', '
   2015-04-01'],
 ['EMP1002', 'Marketing', 'Non-binary', 30, 90000.0, 34, 2.5, 0.39, '
    Active', '2016-12-01'],
 ['EMP1003', 'HR', 'Non-binary', 37, 90000.0, 30, 3.3, 0.01, 'Active',
     '2023-02-01']]
```

Function signature:

```
def remove_null_salaries(data):
    """Remove all records where Salary (index 4) is NaN or missing."""
    return
```

Requirements:

- Check if Salary column (index 4) is `nan`. To do so, make use of `math.isnan(x)`, where `x` is the value you want to check.
- Remove the entire employee record if salary is `nan`
- Return the list of removed entries

4.2.2 Standardize Departments (8+2=10)

Write a function that takes the dataset as an argument, converts all department names to lowercase, and returns nothing.

Sample Input

```
[['EMP1000', 'HR', 'Male', 46, 140000.0, 31, 3.7, 0.23, 'Resigned', '
   2015-04-01'],
 ['EMP1002', 'Marketing', 'Non-binary', 30, 90000.0, 34, 2.5, 0.39, '
    Active', '2016-12-01'],
 ['EMP1003', 'HR', 'Non-binary', 37, 90000.0, 30, 3.3, 0.01, 'Active',
     '2023-02-01']]
```

Sample Output

```
[['EMP1000', 'hr', 'Male', 46, 140000.0, 31, 3.7, 0.23, 'Resigned', '
   2015-04-01'],
 ['EMP1002', 'marketing', 'Non-binary', 30, 90000.0, 34, 2.5, 0.39, '
    Active', '2016-12-01'],
 ['EMP1003', 'sales', 'Non-binary', 37, 90000.0, 30, 3.3, 0.01, '
    Active', '2023-02-01']]
```

Function signature:

```
def standardize_departments(data):
    """Convert all Department names (index 1) to lowercase."""
    pass
```

Requirements:

- Department names may be in various cases (lowercase, UPPERCASE, Mixed)
- Convert all to lowercase (e.g., ‘’engineering’‘, ‘’sales’‘, ‘’hr’‘, ‘’marketing’‘, ‘’product’‘)
- Modify the original list in place

4.2.3 Remove Invalid Performance Ratings (8+2=10)

Write a function that takes the data set as an argument, removes entries that have invalid performance ratings, and returns the data set without the invalid entries. All invalid ratings will have a value outside of the range 0-5, i.e., less than 0 or greater than 5.

Sample Input

```
[['EMP1000', 'HR', 'Male', 46, 140000.0, 31, 3.7, 0.23, 'Resigned', '
   2015-04-01'],
 ['EMP1001', 'Engineering', 'Male', 45, 250000.0, 16, 0.7, 0.08, '
    Resigned', '2016-10-01'],
 ['EMP1002', 'Marketing', 'Non-binary', 30, 90000.0, 34, 25, 0.39, '
    Active', '2016-12-01'],
 ['EMP1003', 'HR', 'Non-binary', 37, 90000.0, 30, 3.3, 0.01, 'Active',
     '2023-02-01']]
```

Sample Output

```
[['EMP1000', 'HR', 'Male', 46, 140000.0, 31, 3.7, 0.23, 'Resigned', '
   2015-04-01'],
 ['EMP1001', 'Engineering', 'Male', 45, 250000.0, 16, 0.7, 0.08, '
    Resigned', '2016-10-01'],
 ['EMP1003', 'HR', 'Non-binary', 37, 90000.0, 30, 3.3, 0.01, 'Active',
     '2023-02-01']]
```

Function signature:

```
def remove_invalid_performance_ratings(data):
    """Remove records with Performance_Rating (index 6) outside range
       [0, 5]."""
    return
```

Requirements:

- Performance ratings must be numeric values between 0 and 5 (inclusive)
- Remove any record with a rating outside this range
- Return the list of removed entries

4.2.4 Fix Hire Dates (8+2=10)

Fix formatting **(3+1=4)**

Write a function that takes the dataset as an argument, fixes the formatting of any hire dates which are not in the required formatting, and returns nothing.

Sample Input

```
[['EMP1000', 'hr', 'Male', 46, 140000.0, 31, 3.7, 0.23, 'Resigned', '
   2015-04-01'],
 ['EMP1002', 'marketing', 'Non-binary', 30, 90000.0, 34, 2.5, 0.39, '
    Active', '01/12/2016'],
 ['EMP1003', 'sales', 'Non-binary', 37, 90000.0, 30, 3.3, 0.01, '
    Active', '2023-02-01']]
```

Sample Output

```
[['EMP1000', 'hr', 'Male', 46, 140000.0, 31, 3.7, 0.23, 'Resigned', '
   2015-04-01'],
 ['EMP1002', 'marketing', 'Non-binary', 30, 90000.0, 34, 2.5, 0.39, '
    Active', '2016-12-01'],
 ['EMP1003', 'sales', 'Non-binary', 37, 90000.0, 30, 3.3, 0.01, '
    Active', '2023-02-01']]
```

Function signature:

```
def fix_format_dates(data):
    """Fix Hire_Date (index 9) format."""
```

```
    pass
```

Requirements:

- Expected format: YYYY-MM-DD
- Some dates may be in DD/MM/YYYY format; convert these to YYYY-MM-DD
- Modify the original list in place

Remove invalid dates **(5+1=6)**

Write a function that takes the dataset as an argument, removes any entries with invalid hire dates, and returns the dataset without the invalid entries.

Sample Input

```
[['EMP1000', 'hr', 'Male', 46, 140000.0, 31, 3.7, 0.23, 'Resigned', '
   2014-04-01'],
 ['EMP1002', 'marketing', 'Non-binary', 30, 90000.0, 34, 2.5, 0.39, '
    Active', '2023-02-01'],
 ['EMP1003', 'sales', 'Non-binary', 37, 90000.0, 30, 3.3, 0.01, '
    Active', '2022-02-29'],
 ['EMP1004', 'engineering', 'Female', 40, 180000.0, 31, 4.0, 0.23, '
    Active', '2016-13-05'],
 ['EMP1005', 'hr', 'Female', 30, 90000.0, 34, 2.5, 0.39, 'Active', '
    2021-03-05'],
 ['EMP1006', 'sales', 'Non-binary', 37, 90000.0, 30, 3.3, 0.01, '
    Active', '2023-02-40']]
```

Sample Output

```
[['EMP1002', 'marketing', 'Non-binary', 30, 90000.0, 34, 2.5, 0.39, '
   Active', '2023-02-01'],
 ['EMP1005', 'hr', 'Female', 30, 90000.0, 34, 2.5, 0.39, 'Active', '
    2021-03-05']]
```

Function signature:

```
def remove_invalid_dates(data):
    """Remove invalid Hire_Date entries(index 9)."""
    return
```

Requirements:

- Hire dates should be between the range of 2015-01-01 and 2025-12-31.
- Some dates are outside of this range or invalid.
- Remove records where:
  - The year is before 2015 (the company was founded in 2015).
  - The year is in the future (after 2025-12-31).
  - The date is logically invalid, that is,
    - Days that are less than 1 or greater than 30. You can assume that every month except February ends on the 30th day.
    - For February specifically, you need to check if the year is a leap year or not. If it is, every day-month combination from February 1 until February 29 is correct, else only February 1 until February 28 is correct.
    - Months that are less than 1 or greater than 12.
- Return the list of removed entries.

4.3 Part B: Metrics Calculation (31+9=40)

Implement nine metric calculation functions. Each function takes an already cleaned dataset as input and returns a calculated result. That is, you do not need your implementation from Part A to do Part B, since the data used for testing will already be cleaned.

NOTE: Round all your function return results to 2 decimal places. That is, use $round(_,2)$ **for any numerical values that are returned from your functions. Ensure that you only round your final/return value**

4.3.1 Get Unique Departments (1+1=2)

Write a function that takes a dataset as an argument, gets the unique departments in the dataset, and returns a set of the unique departments.

Function signature:

```
def get_unique_departments(data):
    """Return a set of all unique department names in the dataset."""
    return
```

Returns: A set of department names

Example: `{'engineering', 'sales', 'hr', 'marketing', 'product'}`

4.3.2 Get Gender Distribution per Department (5+1=6)

Write a function that takes a dataset as an argument, calculates the gender distribution per department in the dataset, and returns a dictionary of dictionaries.

Function signature:

```
def get_gender_distribution(data):
    """Return gender distribution (as percentages) for each department
       ."""
    return
```

Returns: A dictionary where each key is a department name and each value is a dictionary of gender percentages

Example:

```
{
    'engineering': {'Male': 45.5, 'Female': 35.2, 'Non-binary': 19.3},
    'sales': {'Male': 40.0, 'Female': 50.0, 'Non-binary': 10.0},
    ...
}
```

4.3.3 Get Average Age per Department (3+1=4)

Write a function that takes a dataset as an argument, calculates the average age per department for all departments in the dataset, and returns a dictionary.

Function signature:

```
def get_avg_age_by_department(data):
    """Return average age for each department."""
    return
```

Returns: A dictionary where each key is a department name and each value is the average age

Example: `{'engineering': 38.5, 'sales': 35.2, ...}`

4.3.4 Calculate Retention Rate (3+1=4)

Write a function that takes a dataset as an argument, calculates the retention rate of the company, and returns a dictionary.

Function signature:

```
def get_retention_rate(data):
    """Calculate overall retention rate as a percentage."""
    return
```

Returns: A single float representing the retention rate (0-100)

Formula:

$$(Active\ Employees \div Total\ Employees) \times 100$$

Note: Status column (index 8) contains ''Active'' or ''Resigned''. You can calculate the total number of employees by adding the active and resigned employees.

4.3.5 Calculate Turnover Rate per Department (5+1=6)

Write a function that takes a dataset as an argument, calculates the turnover rate of each department for all the departments in the dataset, and returns a dictionary.

Function signature:

```
def get_turnover_rate_by_department(data):
    """Calculate turnover rate for each department as a percentage."""
    return
```

Returns: A dictionary where each key is a department name and each value is the turnover rate

Formula (per department):

$$(Resigned\ Employees\ in\ Department \div Total\ Employees\ in\ Department) \times 100$$

Example: `{'engineering': 15.5, 'sales': 22.3, ...}`

4.3.6 Get Average Salary for Age Range (5+1=6)

Write a function that takes a dataset, a minimum age and a maximum age as arguments, calculates the average salary for the given age range in the dataset, and returns a float.

Function signature:

```
def get_avg_salary_by_age_range(data, min_age, max_age):
    """Return average salary for employees within an age range (
       inclusive)."""
    return
```

Parameters: `min_age` (int), `max_age` (int) — inclusive bounds

Returns: A single float representing the average salary. Account for when no employees are found between a range, that is, when the number of employees in the given range is 0. Your function should return 0 in this case.

4.3.7 Get Average Performance Rating by Training Hours Range per Department (7+1=8)

Write a function that takes a dataset, a minimum training hour value and a maximum training hour value as arguments, calculates the average performance rating for the given training hour range for each department in the dataset, and returns a dictionary.

Function signature:

```
def get_avg_dept_performance_by_training_range(data, min_hours,
   max_hours):
    """Return average performance rating for each department within
       training hours range (inclusive)."""
    return
```

Parameters: `min_hours` (int), `max_hours` (int) — inclusive bounds

Returns: A dictionary representing the average performance rating for each department within a range of training hours. If a department has no employees in the range of the given training hours, set the performance to 0 for that department.

Example:

```
{
    'engineering': 4.2,
```

```
    'sales': 3.9,
    ...
}
```

5 Skeleton Code

You will be provided with skeleton code on Moodle that includes:

- CSV file reading and data parsing into a 2D list.
- Function signatures with docstrings for all functions in Part A and Part B.
- A main guard and sample code demonstrating how to use your functions.

You are not required to use this skeleton code and can implement your own code, but take note of the following if you do so:

- Your function signatures must be identical to the functions in the skeleton code file since the code section will be marked using the Pytest script.
- Your functions must take as input the same data as the functions in the skeleton code file, and do the same thing that these functions do. That is, if a function in the skeleton code returns a dictionary, your equivalent function should do the same so as to be compatible with the test script.

Due to the level of complexity of making your own code implementation, I strongly recommend making use of the skeleton code.

6 Understanding the Main Guard

The main guard is the `if __name__ == "__main__":` statement at the end of your script. Here's how it works:

When Python runs a script directly (e.g., `python solution.py`), Python sets a special variable called `__name__` to the value `"__main__"`. Code inside the if block only runs when the script is executed

directly. However, when a test file (pytest) imports your script, `__name__` is set to the module name (not `"__main__"`), so the code inside the if block is skipped.

Why is this important? It allows pytest to import your functions without executing your sample code. Your submission will be tested with pytest, which will call your functions with different test data.

7 Theoretical Component

You must submit a written report in PDF format. The report should explain your implementation and address the points below.

7.1 Grading Breakdown

Section	Weight
Data Cleaning Explanation	5/20
9 Metrics Explanations	5/20
Alternatives, Tradeoffs & Performance	5/20
Document Quality, Formatting, Neatness	5/20

7.2 Report Structure

7.2.1 Group details

Include your group number, each group member's name, surname, and student number. This is required for marking since only one group member submits.

7.2.2 Section 1: Data Cleaning (7)

For each of the 4 data cleaning tasks, explain:

- What data quality issue you addressed and why it matters
- Your implementation approach

7.2.3 Section 2: Metrics (8)

For each of the 9 metrics, explain:

- Your implementation approach

8 Constraints and Guidelines

8.1 Python Libraries

You may only use the standard library and the `math` module. You cannot use datetime, pandas, numpy, or any other external libraries. This means you must implement all data manipulation using basic Python features (lists, dictionaries, sets, loops, conditionals). That is, content from this course.

NOTE: The use of any other library than the allowed libraries will disqualify your submission and result in a 0 for the sections of your code that make use of any functions from non-approved libraries.

8.2 Code Structure

- Include comments explaining your code. These will be marked as part of the practical section.
- Use helper functions if it makes your code clearer. Code neatness will be considered for the neatness component of the theoretical component.
- Always use the main guard when manually evaluating your code.

8.3 Testing

Before testing your code, have a look at the document on Moodle for testing using PyTest as well as the **TEST_README.pdf** included in the project files. Your submission will be tested with pytest using the provided dataset. Each function will be called with test data, so ensure your functions are general and not hardcoded for specific inputs. Your functions must handle different dataset sizes correctly.

Note that the test file you use to check your submission is the same test file we will use to mark.

9 Submission Requirements

Submit the following:

1. `cleaner.py` — Your complete Python solution for Part A.
2. `metrics.py` — Your complete Python solution for Part B.
3. `solution_report.pdf` — Your report

Ensure your Python file:

- Runs without errors when imported by pytest
- Has comments
- Includes the main guard

Compress these two files into a ZIP file named **Group_number.zip**, and replace **number** with your actual group number. For example, **Group_5.zip**

NOTE: Only one group member should submit the ZIP file on Moodle.

10 Grading Rubric

Component	**Marks**
Data Cleaning Functions (4 functions)	40
Metric Functions (9 functions)	40
Written Report (3 sections)	20
TOTAL	**100**

10.1 Practical Component: Function Breakdown

10.1.1 Cleaning Functions

Each cleaning function is worth 10 marks. Assessment criteria:

- **Correctness:** Does it correctly identify and handle the data quality issue?
- **Code quality:** Is the code readable and well-commented?

10.1.2 Metric Functions

Each metric function is worth a different number of marks. For the weightings, refer to each metric question. Assessment criteria:

- **Correctness:** Does it calculate the metric accurately?
- **Output format:** Does it return data in the specified format?
- **Code quality:** Is the code readable and well-commented?

11 Frequently Asked Questions

11.1 Q: What is *nan*?

A: The provided CSV reading script handles invalid numeric strings by converting them to `float('nan')`. *nan* stands for "not a number". It is stored as `nan` and can be detected by using `math.isnan(x)`, where `x` is the value you want to test for.

11.2 Q: Can I use classes in my solution?

A: Yes, you may use object-oriented programming, but it is not required. The course has not focused on OOP, so functional solutions are perfectly acceptable.

11.3 Q: How should I handle dates that are already in the correct format?

A: Your `fix_hire_dates` function should validate and keep dates that are already in YYYY-MM-DD format if they fall within the valid range (2015 onwards, not in the future).

11.4 Q: How many people in my group should submit?

A: Only one group member should submit to Moodle.

11.5 Q: For metrics that return dictionaries, what order should the departments be in?

A: Dictionary order doesn't matter for testing. The test suite will check that your dictionary contains the correct keys and values, regardless of order.

12 Good Luck!

This project tests your ability to manipulate data, solve real-world problems, and communicate your approach. Write clean code, test thoroughly, and explain your thinking in the report. Reach out if you have questions!